

RÉSEAUX RÉCURRENTS

SOMMAIRE

1-	Définition	1
2-	Entraînement des réseaux récurrents	2
3-	Quelques architectures	4
	3.1- LSTM	
	3.2- GRU	
	3.3- Réseaux récurrents bidirectionnels	
	3.4- Machines de Turing neuronales	
4-	Quelques applications	7
5-	Partie pratique	9
	5.1- Classification de séquences	
	5.2- Prévission de séries temporelles	

1- DÉFINITION

Les réseaux de neurones récurrents (RNN, *Recurrent Neural Networks*) sont des réseaux à propagation avant, permettant de prendre en compte le temps. Comme dans les réseaux classiques, il n'existe pas de cycle, mais les arcs ajoutés pour introduire la notion de temps (les arcs récurrents) peuvent en revanche former des cycles, y compris de longueur 1 (connexion d'un neurone avec lui-même). À l'instant t , les neurones possédant des arcs récurrents reçoivent en entrée la donnée courante $\mathbf{x}_t$ et les valeurs des neurones cachés h_{t-1} informant sur l'état précédent du réseau. La sortie $\hat{y}_t$ est calculée étant donné l'état $\mathbf{x}_t$ des neurones cachés à l'instant t . La donnée $\mathbf{x}_{t-1}$ peut influencer $\hat{y}_t$ et la sortie aux instants suivants, à l'aide des arcs récurrents.

Deux équations permettent de calculer les quantités nécessaires à l'instant t dans la phase de propagation avant d'un réseau récurrent simple (comme celui de la figure 1-1 gauche) :

$$h_t = \sigma \left(\mathbf{W}_{hx}^\top \mathbf{x}_t + \mathbf{W}_{hh}^\top \mathbf{x}_{t-1} + b_h \right)$$

$$\hat{y}_t = \text{softmax} \left(\mathbf{W}_{yh} h_t + b_y \right)$$

où $\mathbf{W}_{hx}$ est la matrice des poids reliant l'entrée à la couche cachée et $\mathbf{W}_{hh}$ celle des poids des arcs récurrents. Les biais sont notés b_h et b_y .

La dynamique du réseau peut être décrite en dépliant ce réseau dans le temps (figure 1-1 droite). Le réseau devient donc un réseau profond, avec une couche par instant t et un partage de poids au cours du temps. Ce dernier peut donc être entraîné de manière classique par l'algorithme de rétropropagation du gradient, indicé par le temps (*Backpropagation through time*, BPTT algorithm).

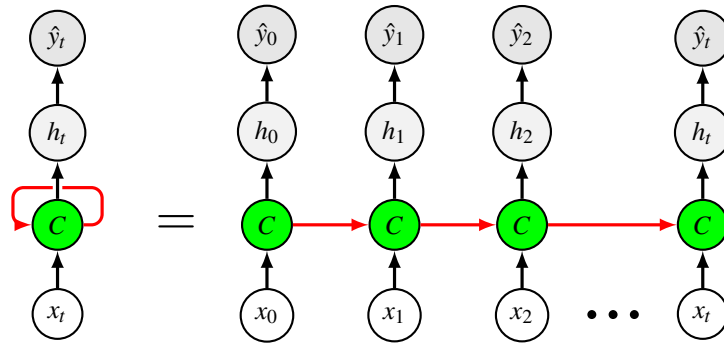


FIGURE 1-1 – Réseau récurrent et sa version dépliée dans le temps.

Avec ces réseaux, il est possible de traiter des séquences de longueur quelconque, la taille du modèle étant indépendante de cette longueur. Plusieurs architectures peuvent être déclinées sur ce principe et le tableau .1 donne un panorama de certaines d'entre elles, avec des exemples d'applications.

2- ENTRAÎNEMENT DES RÉSEAUX RÉCURRENTS

L'apprentissage de dépendances long terme peut être difficile. Les problèmes d'évanescence (*vanishing*) ou d'explosion du gradient peuvent rapidement survenir, lors de la rétropropagation sur plusieurs pas de temps. Prenons un exemple simple pour comprendre : considérons un réseau à un neurone d'entrée, un neurone récurrent caché et un neurone de sortie. On donne au réseau une entrée à l'instant t_0 et on calcule l'erreur à l'instant $t > t_0$, en supposant des entrées nulles entre t_0 et t . Le lien entre les poids au cours du temps fait que le poids sur l'arc récurrent ne change jamais. La contribution de l'entrée au temps t_0 à la sortie au temps t deviendra de plus en plus importante, ou se rapprochera de zéro, de manière exponentielle à mesure que $t - t_0$ croît. Et la dérivée de l'erreur par rapport à l'entrée explosera ou disparaîtra, selon que le poids de l'arc récurrent a une valeur absolue plus grande ou plus petite que 1 et selon la fonction d'activation du neurone caché (le problème du gradient évanescant est très présent avec une sigmoïde et une activation ReLU force davantage l'explosion).

Plusieurs solutions ont été proposées (régularisation, rétropropagation tronquée, conception d'architecture et heuristiques) pour résoudre ces problèmes.

Architecture	Réseau	Applications
Un vers plusieurs		Génération de musique, légendage d'images
Plusieurs vers un		Classification de sentiments
Plusieurs vers plusieurs		Reconnaissance d'entité dans des textes, annotation de vidéos
Plusieurs vers plusieurs		Traduction

TABLE .1 – Différentes architectures de réseaux récurrents et leurs applications

3- QUELQUES ARCHITECTURES

3.1- LSTM

Les réseaux *Long Short-Term Memory* (LSTM) ont été introduits en 1997 [4] pour résoudre le problème de l'évanescence du gradient. Ce modèle ressemble à un réseau récurrent classique à une couche cachée, mais chaque neurone de la couche cachée est remplacé par une cellule de mémoire.

Dans la suite, on note $\mathbf{x}_t$ l'entrée de la cellule à l'instant t , h_{t-1} la sortie de la couche cachée calculée au temps $t-1$. Au lieu de calculer une sortie du type $\sigma(\mathbf{W}^\top \mathbf{x} + b)$, la cellule contient plusieurs éléments distincts aux fonctions particulières. Les LSTM introduisent la notion de portes, qui sont des unités d'activation de type sigmoïde qui prennent comme arguments $\mathbf{x}_t$ et h_{t-1} et viennent pondérer des valeurs calculées dans la cellule. En particulier, si la valeur d'une porte est nulle, alors le flot est coupé dans le graphe, alors qu'il transite intégralement si la valeur de la porte est égale à 1.

On retrouve dans une cellule (figure 3-3) les éléments suivants :

- *Neurone d'entrée* : ce neurone prend en entrée $\mathbf{x}_t$ et h_{t-1} et calcule, à la manière d'un neurone classique, une sortie $g^t = \sigma(\mathbf{W}_C^\top [\mathbf{x}_t, h_{t-1}] + b_C)$.
- *Porte d'entrée* (ou de mise à jour) : la porte calcule $i^t = \sigma(\mathbf{W}_i^\top [\mathbf{x}_t, h_{t-1}] + b_i)$ et vient pondérer la valeur du neurone d'entrée pour décider de l'importance à lui donner au temps t .
- *Porte d'oubli* : cette porte calcule $f^t = \sigma(\mathbf{W}_f^\top [\mathbf{x}_t, h_{t-1}] + b_f)$ et permet au réseau d'oublier son état interne.
- *État interne* : le cœur de la cellule de mémoire est son état interne, noté C^t , composé d'un neurone récurrent à poids fixe unité, assurant que le gradient peut passer par cet arc de nombreuses fois sans disparaître ou exploser. La mise à jour de l'état interne est effectuée par une opération du type $C^t = g^t \cdot i^t + C^{(t-1)} \cdot f^t$.
- *Porte de sortie* : la valeur h_t produite par la cellule de mémoire est calculée comme le produit de $\tanh(C^t)$ par la valeur de la porte de sortie o^t . Cette porte sélectionne la part de C^t à fournir en sortie et est calculée par $o^t = \sigma(\mathbf{W}_o^\top [\mathbf{x}_t, h_{t-1}] + b_o)$.

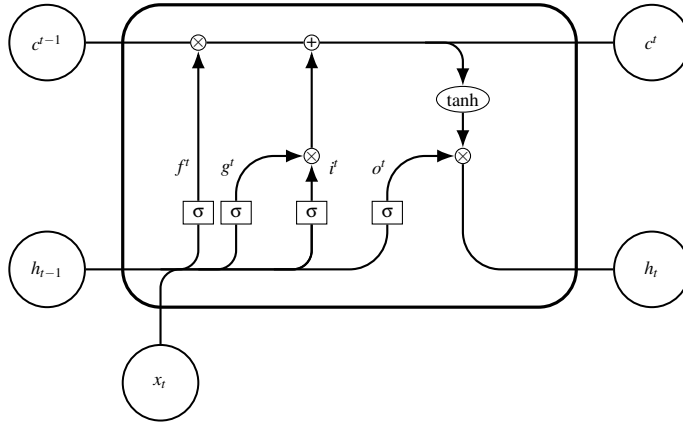


FIGURE 3-2 – Cellule LSTM

En résumé, un LSTM effectue donc les opérations suivantes à l'instant t :

$$\begin{aligned}
g^t &= \sigma(\mathbf{W}_C^\top [\mathbf{x}_t, h_{t-1}] + b_C) \\
i^t &= \sigma(\mathbf{W}_i^\top [\mathbf{x}_t, h_{t-1}] + b_i) \\
f^t &= \sigma(\mathbf{W}_f^\top [\mathbf{x}_t, h_{t-1}] + b_f) \\
o^t &= \sigma(\mathbf{W}_o^\top [\mathbf{x}_t, h_{t-1}] + b_o) \\
C^t &= g^t \cdot i^t + C^{t-1} \cdot f^t \\
h_t &= o^t \tanh(C^t)
\end{aligned}$$

3.2- GRU

En 2014 [1], une version simplifiée des réseaux LSTM a été introduite, qui nécessite moins de paramètres. Les GRU (*Gated Recurrent Units*) sont en effet des réseaux sans mémoire interne C^t , ni porte de sortie o^t . Ces réseaux sont composés de deux portes au lieu de trois :

- une *porte reset* r^t , qui détermine la manière de combiner la nouvelle entrée au temps t avec la mémoire provenant du temps $t-1$.
- une *porte de mise à jour* z^t , qui détermine la quantité de mémoire précédente qui doit être conservée. Cette porte est la combinaison des portes d'entrée et d'oubli des LSTM.

Formellement :

$$\begin{aligned}
r^t &= \sigma(\mathbf{W}_r^\top [\mathbf{x}_t, h_{t-1}] + b_r) \\
z^t &= \sigma(\mathbf{W}_z^\top [\mathbf{x}_t, h_{t-1}] + b_z) \\
\tilde{h}^t &= \tanh(\mathbf{W}^\top [\mathbf{x}_t, r^t h_{t-1}] + b_h) \\
h_t &= (1 - z^t) h_{t-1} + z^t \tilde{h}^t
\end{aligned}$$

Si, pour tout t , $r^t = 1$ et $z^t = 0$, alors on modélise un réseau récurrent classique.

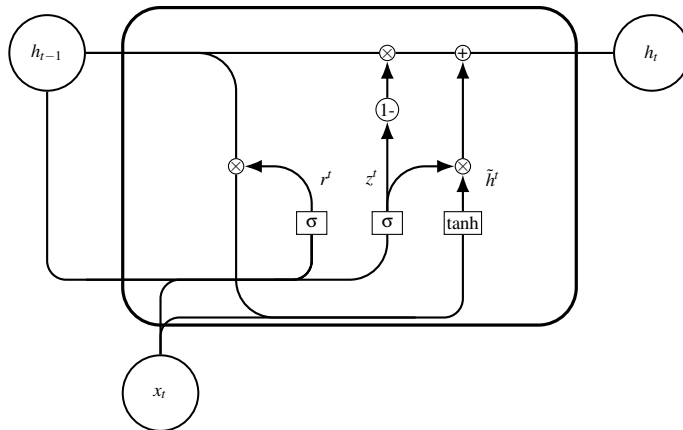


FIGURE 3-3 – Cellule GRU

3.3- Réseaux récurrents bidirectionnels

Les réseaux bidirectionnels ont été décrits pour la première fois en 1997 [5]. Dans ces réseaux, deux couches cachées sont présentes, chacune connectée à l'entrée et la sortie. La première couche cachée a

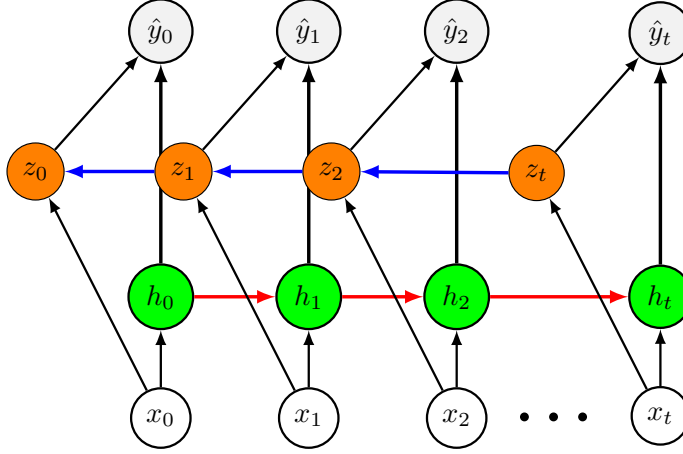


FIGURE 3-4 – Réseau bidirectionnel.

des connexions récurrentes depuis le passé vers le futur, tandis que l'autre transmet les activations depuis le futur vers le passé (figure 3-4).

Étant données une entrée et une sortie du réseau (des séquences), le réseau peut être entraîné par rétropropagation après avoir été déplié :

$$\begin{aligned}x_t &= \sigma \left(\mathbf{W}_h^\top [\mathbf{x}_t, h_{t-1}] + b_h \right) \\z_t &= \sigma \left(\mathbf{W}_z^\top [\mathbf{x}_t, z_{t+1}] + b_z \right) \\y_t &= \text{softmax} \left(\mathbf{W}_y^\top [x_t, z_t] + b_y \right)\end{aligned}$$

où h_t (respectivement z_t) représente la valeur de la couche cachée dans le sens du temps (respectivement dans le sens inverse). Puisque le temps doit être fini dans les deux sens de parcours, les réseaux bidirectionnels ne peuvent traiter que des séquences finies.

3.4- Machines de Turing neuronales

Les réseaux récurrents sont performants pour construire une représentation implicite de l'information, mais restent relativement peu adaptés à la conservation d'informations explicites (des dates précises par exemple). S'inspirant des mémoires de travail, théorisées par les neurosciences et qui sont responsables du raisonnement inductif et de la création de nouveaux concepts, l'idée est alors d'ajouter à ces modèles une « mémoire de travail » externe, ce qui permet de découpler la mémoire (assimilable à la RAM d'un ordinateur) des opérations liées à la tâche effectuée par le réseau (assimilable à la CPU). Puisque la mémoire des LSTM est distribuée dans chaque cellule, elle est donc liée au nombre de cellules et à la capacité de calcul et ce modèle ne répond pas directement au problème posé.

Graves et al. [3] proposent alors une architecture, appelée machine de Turing neuronale, constituée de deux éléments principaux : une mémoire et un contrôleur doté d'un mécanisme d'attention qui lit et écrit dans cette mémoire. Les accès mémoire sont ici des équivalents analogiques dérivables, pour permettre d'entraîner le contrôleur par descente de gradient. Typiquement, le contrôleur est un réseau de neurones ou un réseau récurrent type LSTM (figure 3-5).

Les têtes de lecture et d'écriture interagissent avec la mémoire. Chaque tête est contrôlée par un vecteur de poids, chaque composante définissant le degré d'interaction de la tête avec la zone mémoire correspondante. Un *mécanisme de mise à jour de ces poids*, composé de quatre opérations, est mis en place pour permettre l'apprentissage du réseau :

1. Le réseau s'intéresse tout d'abord aux zones mémoire proches d'une clé k_i donnée. Cela permet au modèle de retrouver une information spécifique, en recherchant si la zone mémoire $M_i(i)$ est proche

de la clé, au sens d'une similarité K . Formellement, chaque poids correspondant à la zone mémoire i est calculé par $w_t(i) = \text{softmax}(\beta_t K[k_t, M_t(i)])$.

2. Un mécanisme d'interpolation linéaire permet ensuite de mettre à jour les poids en fonction de leur valeur précédente (pour prendre plus ou moins en compte l'information issue de la clé, ou au contraire la valeur précédente du poids) : $w_t(i) = g_t \cdot w_t(i) + (1 - g_t) \cdot w_{t-1}(i)$.
3. Un décalage par convolution translate ensuite les poids, à la manière du décalage classique de la tête dans une machine de Turing classique : $w_t(i) = \sum_j w_t(j) \mathbf{s}_t(i - j)$ où $\mathbf{s}_t$ est un vecteur qui définit un décalage des poids à l'instant t .
4. Enfin, le vecteur de poids est focalisé : $w_t(i) = w_t(i)^{\gamma_t}$, $\gamma_t > 1$.

Une fois que la tête a mis à jour les poids, elle interagit avec la mémoire :

- Dans le cas de la tête de lecture, elle calcule une combinaison linéaire des zones mémoire, pondérées par les poids $w_t(i)$, et produit le vecteur $\mathbf{r}_t$, fourni au contrôleur de l'instant suivant.
- Dans le cas de la tête d'écriture, le contenu de la mémoire est mis à jour selon la formule $M_t(i) = M_{t-1}(i)(1 - w_t(i)\mathbf{e}_t) + w_t(i)\mathbf{a}_t$, où $\mathbf{e}_t$ est un vecteur d'effacement, dont les composantes sont dans $\{0,1\}$ et $\mathbf{a}_t$ est un vecteur d'ajout.

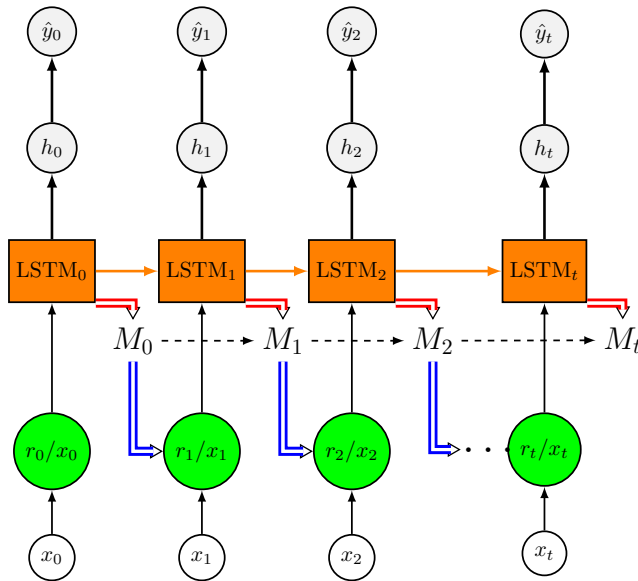


FIGURE 3-5 – Exemple de machine de Turing neuronale dépliée dans le temps, où le contrôleur est un LSTM. Les accès en écriture du LSTM dans la mémoire sont représentés par des flèche rouges, les accès en lecture en bleu.

4- QUELQUES APPLICATIONS

Comme les réseaux convolutifs, les réseaux récurrents ont depuis leur introduction trouvé de nombreuses applications.

Traitement automatique du langage

Les réseaux récurrents sont utilisés en traitement automatique du langage, notamment à des fins génératives. Ces réseaux permettent de modéliser un langage (prédire la probabilité d'un mot donné étant donnés les mots précédents) et de générer du texte à partir du modèle appris. De nombreuses applications découlent de cette modélisation : génération de texte « au style de » (génération d'un texte dans le style de Shakespeare, à partir d'un RNN appris sur le corpus des œuvres de l'auteur par exemple, génération de textes manuscrits (figure 3-6(a)), génération de pages Wikipedia, ou même génération d'articles scientifiques, à partir des sources $\text{\LaTeX}$ d'un ouvrage et d'un LSTM multicouche.

Traduction automatique

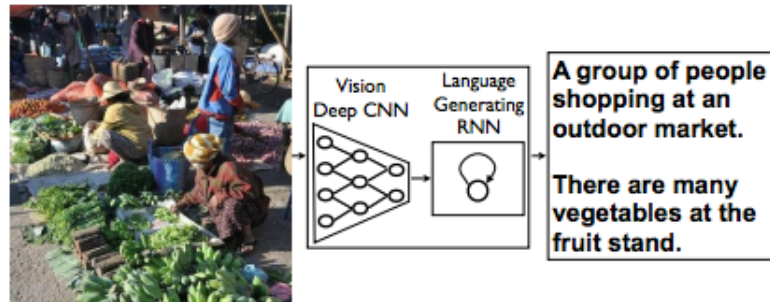
Apprentissage artificiel, concepts et algorithmes

Apprentissage artificiel, concepts et algorithmes

Apprentissage artificiel, concepts et algorithmes

Apprentissage artificiel, concepts et algorithmes.

(a) Génération de texte manuscrit (source [2])



(b) Génération de légendes (source [6])

FIGURE 3-6 – Quelques applications des réseaux récurrents.

La traduction automatique de texte procède de la même stratégie que la modélisation d'une langue. Deux réseaux récurrents sont entraînés, chacun dans une des langues, et le RNN traducteur calcule sa sortie en fonction de la couche cachée du premier réseau.

Analyse de sentiments

Détecter de manière automatique l'opinion du public sur un sujet donné intéresse de plus en plus le domaine commercial. Ce domaine, largement alimenté par les réseaux sociaux, les avis et recommandations déposées sur les sites Internet, est un champ de prédilection pour les réseaux profonds. Des réseaux récurrents (notamment LSTM structurés en arbres) sont utilisés à cet effet et servent de base à des systèmes de recommandation.

Résumé automatique

Les réseaux récurrents permettent de produire des résumés abstraits de textes (i.e. générer de nouvelles phrases, en opposition à extraire les mots les plus importants d'un texte). Les modèles utilisés sont des réseaux récurrents avec mécanisme d'attention. Un système d'encodage/décodage est mis en place dans le réseau, où l'encodeur est par exemple un GRU bidirectionnel et le décodeur un GRU dont l'état caché a la même taille que celui de l'encodeur. Les modèles sont appris et validés sur des corpus dédiés (DUC, CNN/Daily Mail par exemple).

Reconnaissance de la parole

L'utilisation de réseaux LSTM bidirectionnels, qui permettent à la fois d'exploiter les contextes passé et futur, et de garder trace d'un contexte à longue échéance, a montré de bonnes performances dans la tâche de reconnaissance de la parole.

Annotation d'images

Couplé à un réseau convolutif, un RNN permet de générer des descriptions (légendes) d'images non labellisées. Le réseau convolutif produit des descripteurs, qui servent d'entrée à un réseau récurrent type LSTM (figure 3-6(b)).

5- PARTIE PRATIQUE

5.1- Classification de séquences

L'objectif de cette partie est d'apprendre à l'aide de réseaux récurrents à classer des séquences (ici des phrases) en plusieurs classes. Le cas d'étude est une base de données d'avis de films (IMDB) : pour chaque film, un avis est rédigé, et un label positif (1) ou négatif (0) est assigné à cet avis. Il s'agit alors d'apprendre à reconnaître la qualité de l'avis, et de pouvoir assigner un label 0/1 à un nouvel avis, jamais vu par le réseau.

Vous avez à disposition un notebook donnant l'essentiel des codes permettant de :

- lire la base de données
- mettre en forme les données
- proposer un modèle de base auquel comparer vos résultats

Votre travail consiste alors à construire un réseau récurrent permettant d'effectuer la tâche et de le tester.

Pour ceci, vous aurez à :

- utiliser la fonction `keras.preprocessing.sequence.pad_sequences()` pour prétraiter les données d'entraînement. Cette fonction créera un tableau 2D, avec en ligne les 25000 avis et en colonnes les `maxlen` premiers mots de l'avis. Si l'avis est plus long, il sera coupé, si l'avis est plus court il sera complété par des 0.
- construire un réseau avec
 1. une couche d'**Embedding**, à la dimension d'entrée appropriée, et dont la dimension de sortie est égale à 10. Le modèle apprendra donc à représenter chaque mot par un vecteur de $\mathbb{R}^{10}$
 2. une (ou plusieurs) couche(s) **LSTM** à 32 neurones
 3. une couche dense de sortie, à activation sigmoïde (problème de classification binaire)

Le modèle sera compilé avec la fonction de perte adéquate pour un tel problème, et entraîné sur des batchs de taille 128.

- améliorer ce réseau en utilisant un modèle bidirectionnel. Pour ceci, il suffit d'encapsuler la ou les couche(s) LSTM dans une **couche bidirectionnelle**. Si le modèle sur apprend, vous pouvez ajouter une couche de **Dropout**.

5.2- Prévision de séries temporelles

En utilisant l'API Keras, on se propose d'implémenter un RNN permettant de faire de la prédiction de série temporelle. Le notebook `RNN prevision` propose de construire un réseau récurrent simple (sans module LSTM ou GRU) permettant de faire de la prévision météo à partir de données de température, pression et humidité mesurées pendant 5 ans. Les données sont issues d'un challenge [Kaggle](#). Elles sont mises en forme pour un RNN simple, implémenté sous Keras via la couche **SimpleRNN**. Votre travail consiste simplement à :

- définir (fonction `myRNN`) un réseau récurrent comportant une couche `SimpleRNN` et deux couches complètement connectées (figure 5-7).
- Proposer des graphiques de comparaison des courbes réelles et prédites.
- Comparer les prédictions avec celles calculées par des réseaux **LSTM** et **GRU**.

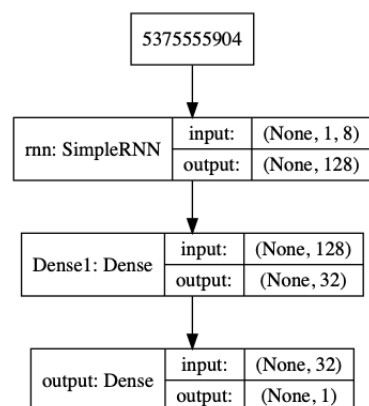


FIGURE 5-7 – Réseau récurrent à réaliser

- [1] J Chung, Ç Gülçehre, K Cho, and Y Bengio. Empirical evaluation of gated recurrent neural networks on sequence modeling. CoRR, abs/1412.3555, 2014.
- [2] A. Graves. Generating sequences with recurrent neural networks. CoRR, abs/1308.0850, 2013.
- [3] A Graves, G Wayne, and I Danihelka. Neural turing machines. CoRR, abs/1410.5401, 2014.
- [4] S. Hochreiter and J. Schmidhuber. Long short-term memory. Neural Comput., 9(8) :1735–1780, November 1997.
- [5] M. Schuster and K.K. Paliwal. Bidirectional recurrent neural networks. Trans. Sig. Proc., 45(11) :2673–2681, November 1997.
- [6] O. Vinyals, A. Toshev, S. Bengio, and D. Erhan. Show and tell : A neural image caption generator. CoRR, abs/1411.4555, 2014.